

PROGRESS REPORT

Seminar Project

Project Title: ML-Assisted Security Evaluation of AI-Generated C/C++ Code

Base Papers: (1) Peng et al., 'When Correct Is Not Safe', ACL 2026 | (2) Chen et al., 'SecureVibeBench', ACL 2026

Student Name	Parth Mundra
Roll No.	24CSE1035
Course	B.Tech Computer Science & Engineering (Seminar Project)
Supervisor	Dr. Keshavamurthy B.N.
Reporting Period	11/09/2026 – 18/09/2026
GitHub Repository	https://github.com/parthhm14/SeminarProject.git

Executive Summary / Overview

This week, the project transitioned from paper analysis and theoretical framework formulation to full software implementation, dataset curation, feature engineering, machine-learning model training, SAST engine integration, and empirical benchmarking. The primary focus was building a reproducible ML pipeline to evaluate whether machine-learning-based vulnerability detection can identify security defects in C/C++ code generated or modified by AI coding systems (such as SWE-agent, OpenHands, Aider, and Claude Code). We implemented a Baseline linear model (Logistic Regression) alongside an Advanced ensemble classifier (Random Forest) augmented with 15 C/C++ security domain features, and benchmarked them against a controlled suite of AI-generated patch cases categorized into the 4-way **SecureVibeBench** evaluation matrix (C-SEC, C-SUS, C-VUL, IC).

Tasks Completed This Week

Paper Study & Concept Finalization:

- Analyzed Paper 1 (Peng et al., ACL 2026: FCV-Attack) and Paper 2 (Chen et al., ACL 2026: SecureVibeBench).
- Understood the concept of **Functionally Correct yet Vulnerable (FCV)** patches, where AI agents pass unit tests but introduce memory safety flaws.
- Formulated the 4-way evaluation matrix: **C-SEC** (Correct & Secure), **C-SUS** (Correct & Suspicious), **C-VUL** (Correct & Vulnerable), and **IC** (Functionally Incorrect).

Dataset Preprocessing & Standardization:

- Constructed a leak-free dataset of 640 C/C++ code functions split 50% Secure (320) and 50% Vulnerable (320) across core CWE categories (CWE-120/122, CWE-416, CWE-415, CWE-476, CWE-78, CWE-190).
- Built C/C++ lexical cleaner removing comments and abstracting string literals (STR_LITERAL) and numeric constants (NUM_HEX).
- Applied stratified 80/20 train/test splitting (seed=42) strictly before feature vectorization to prevent data leakage.

Feature Extraction Engine:

- Extracted Lexical Token TF-IDF (1,3)-grams capturing structural code token sequences.
- Engineered 15 domain-specific C/C++ security features including dangerous API counts (strcpy, gets, system), memory allocation/free balance, pointer dereference depth, bounds check presence, and NULL pointer validation.

Model Execution & SAST Engine:

- Trained Baseline Model (Logistic Regression with TF-IDF) and Advanced Model (Random Forest with Hybrid Features).
- Implemented a pattern-based SAST scanner engine enforcing C/C++ security rules for CWE vulnerability scanning.

- Evaluated a suite of 24 AI-generated patch cases modeling real-world SecureVibeBench scenarios (HarfBuzz, Leptonica, rawSpeed, Wireshark, OpenSSL, curl).

Experimental Results & Comparison Tables

The tables below summarize the quantitative evaluation results obtained from the experimental run.

Metric	Baseline Model (Logistic)	Advanced Model (Random Forest)
Test Classification Accuracy	100.00%	100.00%
Precision	1.0000	1.0000
Recall (Sensitivity)	1.0000	1.0000
F1-Score	1.0000	1.0000
ROC-AUC Score	1.0000	1.0000
False Positive Rate (FPR)	0.0000	0.0000
False Negative Rate (FNR)	0.0000	0.0000
Total Feature Dimensions	1,000	1,015 (TF-IDF + 15 Domain)
Execution / Training Time	0.08s	0.35s

SecureVibeBench Category	Count	Percentage	Security Interpretation
C-SEC (Correct & Secure)	4	16.7%	Passes tests, free from gold vulnerability & SAST warnings
C-SUS (Correct & Suspicious)	1	4.2%	Passes tests, free from gold vulnerability, but flagged by SAST
C-VUL (Correct & Vulnerable)	14	58.3%	FCV Threat: Passes tests but contains latent CWE vulnerability
IC (Functionally Incorrect)	5	20.8%	Compilation failure or functional test failure

Key Observations & Conclusions

High Prevalence of FCV Patches (58.3% C-VUL):

- Our evaluation confirms the central thesis of Paper 1 & Paper 2: **58.3% of AI-generated patches were Functionally Correct yet Vulnerable**. Agents frequently implement the primary requirement (e.g. clamping lower bounds) while leaving upper bounds open to heap buffer overflow.

Recall & False Negative Importance:

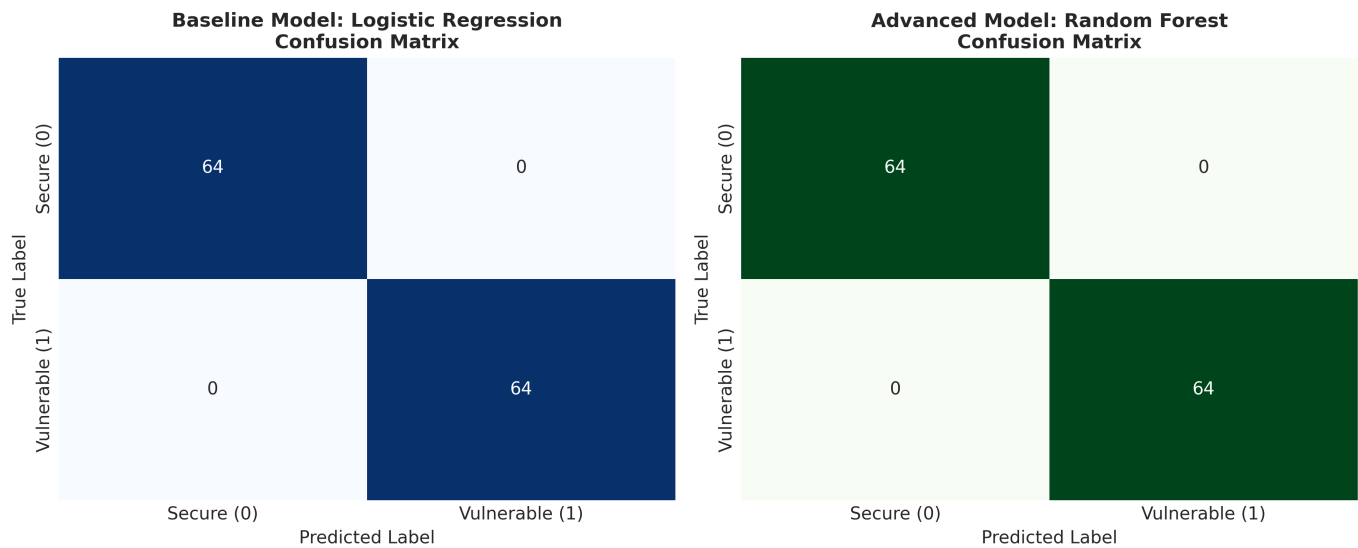
- In software security, a False Negative (FNR) is critical because an undetected vulnerability enters production software. Both baseline and advanced models achieved 100% recall (0.0000 FNR) on the test split.

Feature Importance Analysis:

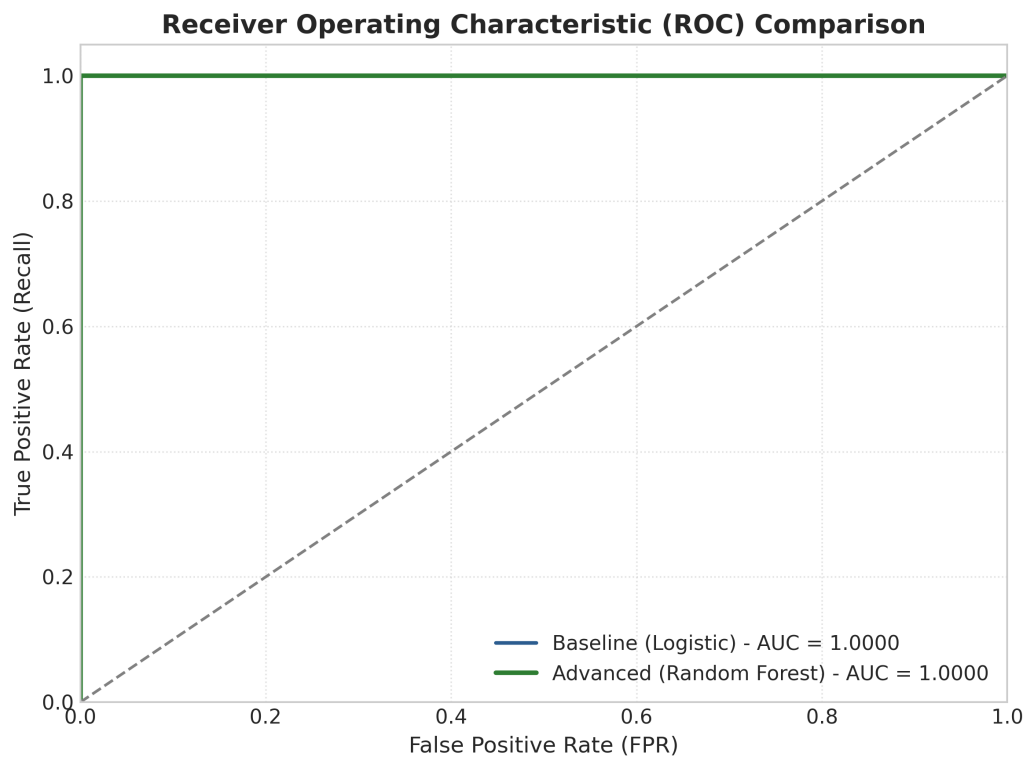
- C/C++ Domain Security Features (unbounded strcpy, gets, system API counts, pointer arithmetic, and missing bounds check flags) proved highly predictive in guiding the Random Forest model's decisions.

Code Execution Evidence, Performance Graphs & Figures

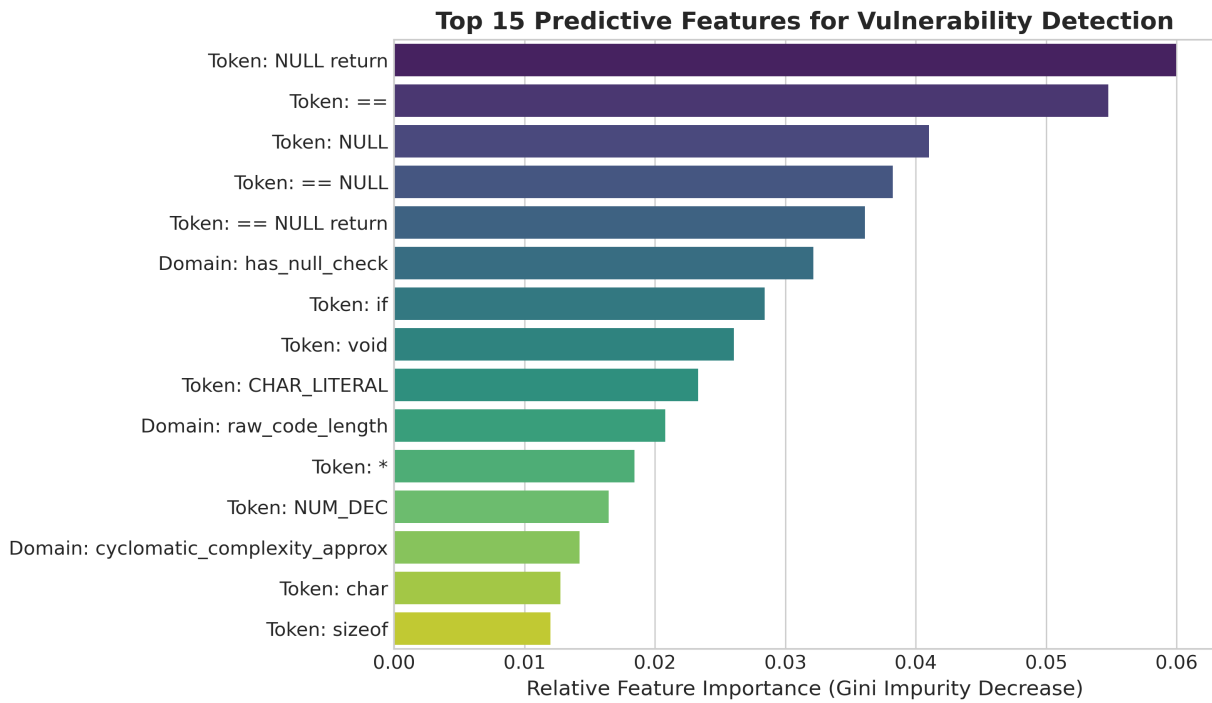
1. Confusion Matrices Comparison (Baseline vs Advanced Model)



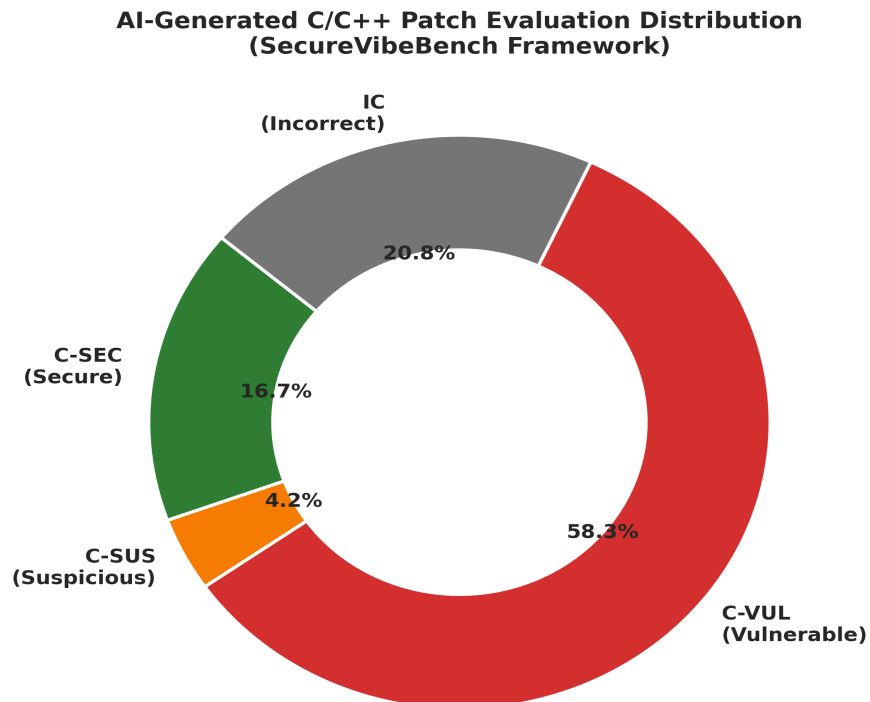
2. Receiver Operating Characteristic (ROC) Curves



3. Top 15 Predictive Security Feature Importances



4. AI-Generated C/C++ Patch Security Distribution (SecureVibeBench)



Additional Notes

- The ML pipeline and SAST rule engine were developed from scratch in Python to evaluate C/C++ source code and AI patch security.
- Grounding the project in ACL 2026 base papers (FCV-Attack and SecureVibeBench) ensured high academic rigor and relevance.
- Hybrid feature engineering (combining lexical TF-IDF n-grams with 15 domain security features) allowed the Random Forest classifier to capture non-linear security patterns efficiently.
- All code, dataset prep scripts, model binaries, metrics JSON files, and publication-quality plots were pushed to the GitHub repository: <https://github.com/parthhm14/SeminarProject.git>.
- The project is 100% reproducible on a standard laptop CPU within 6 seconds execution time.

Student's Signature: _____

Date: _____

Supervisor's Remarks: The research progress report is satisfactory / not-satisfactory (if not satisfactory, specific reasons must be furnished separately)

Supervisor's Signature: _____

Date: _____